Trabalho Prático 1 – TP1

Algoritmos e Classificação de Dados (ACD)

1. Objetivo do trabalho

O TP1 consiste em um **projeto prático de programação, experimentação, análise e apresentação de resultados** envolvendo algoritmos de ordenação estudados no componente de Algoritmos e Classificação de Dados.

O trabalho tem como objetivos:

- implementar diferentes algoritmos de ordenação em Java;
- verificar experimentalmente a corretude das implementações;
- planejar e executar experimentos para avaliar o desempenho dos algoritmos;
- coletar, organizar e analisar os tempos de execução;
- comparar os resultados experimentais com a análise teórica de complexidade dos algoritmos;
- identificar características da implementação e das entradas que possam influenciar o desempenho;
- apresentar e discutir os resultados obtidos em um documento técnico e em uma apresentação.

O trabalho poderá ser realizado em **duplas** e, excepcionalmente, em **trios**. O TP1 terá peso de **16% na média do componente de ACD** e será computado como atividade EaD, com carga horária vinculada de **5 horas**.

O trabalho deverá ser apresentado pelo(s) discente(s) ao docente e, preferencialmente, também à turma, conforme o cronograma do componente.

2. Algoritmos a serem implementados

Cada grupo deverá implementar, em Java, os seguintes algoritmos:

Algoritmos baseados em comparação

- Bubble Sort;
- Insertion Sort;
- Selection Sort;
- Shell Sort;
- Heap Sort;
- Merge Sort;
- Quick Sort.

Algoritmos de tempo linear

- Counting Sort;
- Radix Sort;
- Bucket Sort.

Os algoritmos deverão ser implementados pelos próprios grupos, não sendo permitido utilizar diretamente métodos de ordenação prontos da API Java para substituir as implementações solicitadas.

Para os algoritmos baseados em comparação, recomenda-se a utilização da interface `Comparable` da API Java, permitindo que os métodos trabalhem com diferentes tipos de dados comparáveis.

Por exemplo:

```
public class Ordenadores {

    public static <T extends Comparable<T>>
    void quicksort(T[] a, int lo, int hi) {
        // ...
    }

    public static <T extends Comparable<T>>
    void heapsort(T[] a, int n) {
        // ...
    }

    // demais algoritmos e métodos auxiliares
}
```

Cada método deverá possuir os parâmetros necessários ao seu funcionamento. Métodos auxiliares poderão ser utilizados quando necessários.

Os algoritmos Counting Sort, Radix Sort e Bucket Sort possuem requisitos específicos sobre os dados de entrada e, portanto, deverão ser tratados considerando suas características próprias.

3. Etapas do trabalho

Etapa 1 – Implementação e verificação dos algoritmos

Inicialmente, cada grupo deverá implementar e testar os dez algoritmos de ordenação.

Além da implementação, deverão ser preparados **vetores pequenos de teste**, entre 10 e 15 elementos, para verificar o comportamento dos algoritmos em diferentes situações.

Recomenda-se utilizar, no mínimo, os seguintes casos:

1. elementos em ordem crescente;
2. elementos em ordem decrescente;
3. elementos em ordem aleatória;
4. elementos com várias ocorrências repetidas.

Os vetores deverão possuir tamanhos diferentes, incluindo tamanhos pares e ímpares.

Esses testes não têm como objetivo provar formalmente a corretude dos algoritmos. Seu propósito é permitir uma verificação prática inicial das implementações antes da realização dos experimentos de desempenho.

Etapa 2 – Preparação dos dados de entrada

Os experimentos deverão utilizar vetores com diferentes características.

2.1 Ordem dos elementos

Deverão ser considerados, sempre que aplicável:

- elementos em ordem crescente;
- elementos em ordem decrescente;
- elementos em ordem aleatória.

2.2 Tamanho dos vetores

Sugere-se inicialmente utilizar vetores com tamanhos da ordem de:

- 1.000 elementos;
- 10.000 elementos;
- 100.000 elementos;
- 1.000.000 de elementos.
- …?

Os grupos poderão utilizar tamanhos maiores ou diferentes, desde que os computadores e o tempo disponível permitam a realização dos experimentos.

Para os vetores em ordem crescente e decrescente, poderá ser utilizado apenas um dos maiores tamanhos considerados no experimento, caso isso seja suficiente para a análise.

2.3 Comparabilidade das entradas

Para uma comparação justa entre algoritmos, **todos os algoritmos que participarem de uma determinada configuração experimental deverão receber exatamente os mesmos dados de entrada**.

Como os algoritmos normalmente modificam o vetor durante a ordenação, deverá ser utilizada uma cópia do vetor original para cada execução.

Por exemplo:

```
vetor original
      |
      ├── cópia → Bubble Sort
      ├── cópia → Insertion Sort
      ├── cópia → Selection Sort
      ├── cópia → ...
      └── cópia → Quick Sort
```

Dessa forma, a execução de um algoritmo não altera os dados utilizados pelos demais.

4. Etapa 3 – Medição dos tempos de execução

Cada grupo deverá preparar um código auxiliar responsável por executar os algoritmos e registrar seus tempos de execução.

A medição deverá considerar **somente o processo de ordenação**, não devendo incluir a geração dos vetores, a criação das cópias, a impressão dos resultados ou outras atividades externas ao algoritmo.

Um esquema simplificado é:

```
long inicio = System.nanoTime();

sort(vetor);

long fim = System.nanoTime();

long tempo = fim - inicio;
```

Recomenda-se utilizar o método `System.nanoTime()` da API Java para a medição dos intervalos de tempo.

Como a execução ocorre sobre a máquina virtual Java, recomenda-se realizar algumas execuções preliminares antes da coleta dos dados utilizados nos resultados, de modo a reduzir a influência das primeiras execuções da JVM. Esses tempos preliminares não deverão fazer parte dos resultados apresentados.

5. Etapa 4 – Planejamento e execução dos experimentos

Cada combinação de:

algoritmo × tamanho do vetor × configuração dos dados

deverá ser executada várias vezes.

Deverão ser realizadas **pelo menos 10 execuções** de cada configuração, sendo recomendadas **30 ou mais repetições** quando o tempo disponível permitir.

Todos os tempos individuais deverão ser registrados.

Os experimentos deverão procurar manter condições semelhantes entre as diferentes execuções.

Por exemplo:

- utilizar as mesmas entradas;
- utilizar cópias equivalentes dos vetores;
- medir somente o tempo de ordenação;
- utilizar a mesma configuração de hardware e software;
- evitar, tanto quanto possível, atividades externas que possam interferir significativamente nas medições.

Não é necessário eliminar completamente todas as fontes de variação existentes em um computador real. Entretanto, o grupo deverá reconhecer que fatores externos podem influenciar os tempos medidos e deverá procurar minimizar seus efeitos.

6. Algoritmos de tempo linear

Counting Sort, Radix Sort e Bucket Sort deverão ser analisados considerando suas características específicas.

Esses algoritmos não utilizam comparação entre elementos da mesma forma que os demais algoritmos estudados e dependem de determinadas propriedades dos dados de entrada.

Por esse motivo, os grupos deverão descrever as condições utilizadas para esses algoritmos e discutir as implicações dessas condições para os resultados obtidos.

Sempre que for possível estabelecer uma comparação adequada com os demais algoritmos, essa comparação poderá ser realizada. Nesse caso, deverão ser explicitadas as condições que tornam a comparação válida.

A análise deverá considerar que a complexidade assintótica, isoladamente, não determina o tempo de execução observado em uma implementação concreta.

7. Etapa 5 – Organização e análise dos resultados

Os tempos de todas as execuções deverão ser preservados e organizados para posterior análise.

Para cada configuração experimental, deverão ser calculadas e/ou apresentadas estatísticas adequadas, incluindo, no mínimo, a **média dos tempos de execução**.

Recomenda-se também considerar outras medidas, como:

- mediana;
- mínimo e máximo;
- desvio-padrão ou outra medida de dispersão.

Os resultados deverão ser apresentados por meio de gráficos adequados.

Poderão ser utilizados, entre outros:

- gráficos de barras;
- gráficos de dispersão;
- **boxplots (diagramas de caixa)**.

Os gráficos deverão permitir comparar o comportamento dos algoritmos e identificar a variação observada entre as diferentes execuções.

8. Questões orientadoras para a análise

A análise dos resultados deverá ir além da simples apresentação dos tempos.

Entre as questões que poderão orientar a discussão estão:

1. Os resultados experimentais são compatíveis com as complexidades assintóticas estudadas em aula?
2. Como o tamanho do vetor influencia o tempo de execução de cada algoritmo?
3. O comportamento dos algoritmos muda quando os dados estão ordenados, invertidos ou em ordem aleatória?
4. Os resultados observados correspondem ao comportamento esperado para o melhor, médio e/ou pior caso dos algoritmos?
5. Existem diferenças significativas entre os algoritmos que possuem a mesma ordem de complexidade assintótica?
6. Que características da implementação podem ajudar a explicar as diferenças observadas?
7. Como os algoritmos de tempo linear se comportam nas condições utilizadas nos experimentos?
8. É possível observar, experimentalmente, os efeitos previstos pela análise teórica dos algoritmos?

As respostas deverão ser fundamentadas nos resultados obtidos nos experimentos.

9. Etapa 6 – Relatório técnico

Cada grupo deverá elaborar um **documento técnico**, que poderá assumir a forma de relatório ou artigo científico, descrevendo o experimento realizado.

O documento deverá conter, no mínimo:

9.1 Introdução

Apresentação do problema, dos algoritmos estudados e dos objetivos do experimento.

9.2 Algoritmos analisados

Breve descrição dos algoritmos de ordenação utilizados, incluindo suas principais características e complexidades.

9.3 Metodologia

Descrição suficientemente detalhada para permitir a compreensão de como o experimento foi realizado, incluindo:

- hardware e software utilizados;
- algoritmos avaliados;
- características dos vetores de entrada;
- tamanhos utilizados;
- número de repetições;
- procedimento de medição dos tempos;
- tratamento dos dados coletados.

9.4 Resultados

Apresentação dos tempos obtidos, estatísticas e gráficos.

9.5 Análise e discussão

Interpretação dos resultados e comparação com a análise teórica dos algoritmos.

9.6 Conclusões

Síntese dos principais resultados e das conclusões obtidas a partir dos experimentos.

Recomenda-se a utilização do **Overleaf/LaTeX** (ou outro ambiente de escrita em Latex) para a elaboração do documento. O formato final do relatório ou artigo deverá ser discutido com o docente.

10. Etapa 7 – Apresentação

Cada grupo deverá preparar uma apresentação com duração aproximada de **10 minutos**.

A apresentação deverá abordar, de forma objetiva:

1. o problema e os objetivos do experimento;
2. a metodologia utilizada;
3. os principais resultados;
4. a análise dos resultados;
5. as conclusões.

A apresentação ocorrerá durante as aulas, conforme o cronograma do componente. Quando necessário, poderão ser realizadas apresentações em horário extraclasse.

11. Entregáveis

Ao final do trabalho, cada grupo deverá disponibilizar:

1. Código-fonte

- implementação dos dez algoritmos;
- métodos auxiliares necessários;
- código dos testes de corretude;
- código utilizado para a execução dos experimentos;
- código utilizado para registro dos resultados.

2. Dados experimentais

- tempos individuais de todas as execuções;
- identificação do algoritmo;
- tamanho e configuração da entrada utilizada;
- demais informações necessárias para interpretar os dados.

3. Relatório ou artigo técnico

Documento contendo a metodologia, resultados, análise e conclusões.

4. Apresentação

Material utilizado na apresentação do trabalho.

12. Critérios de análise do trabalho

A avaliação do TP1 considerará principalmente:

- **corretude das implementações** dos algoritmos;
- **organização e qualidade do código**;
- **adequação dos testes realizados**;
- **planejamento e execução dos experimentos**;
- **qualidade e confiabilidade dos dados coletados**;
- **adequação dos gráficos e demais formas de apresentação dos resultados**;
- **capacidade de relacionar os resultados experimentais com a análise de complexidade dos algoritmos**;
- **qualidade da análise e da discussão dos resultados**;
- **qualidade do relatório/artigo técnico**;
- **clareza e objetividade da apresentação**.

Mais importante do que simplesmente obter os menores tempos de execução é demonstrar que o grupo foi capaz de **planejar o experimento, coletar dados confiáveis, interpretar os resultados e relacioná-los aos conceitos estudados no componente**.

13. Escopo do experimento

O escopo do experimento poderá ser descrito utilizando o seguinte modelo:

> **Analyze** <Object(s) of study>
> **for the purpose of** <Purpose>
> **with respect to their** <Quality focus>
> **from the point of view of the** <Perspective>
> **in the context of** <Context>.

Para o TP1:

> **Analisar** os métodos de ordenação de dados
> **para o propósito de** avaliação
> **com respeito ao** tempo de execução e ao comportamento de desempenho
> **do ponto de vista do(a)** programador(a)
> **no contexto de** estudos do componente de Algoritmos e Classificação de Dados.

14. Possibilidades de extensão

Os grupos são incentivados a realizar variações ou extensões dos experimentos, desde que a proposta principal do TP1 seja atendida.

Algumas possibilidades incluem:

- comparar implementações genéricas baseadas em `Comparable` com implementações específicas para tipos primitivos, como `int`;
- investigar a influência do uso de tipos genéricos sobre o desempenho;

- comparar uma implementação modificada de um algoritmo com sua versão original;
- investigar uma combinação de algoritmos como, por exemplo, Merge Sort e Insertion Sort;
- comparar outros algoritmos de ordenação com aqueles estudados;
- investigar métodos como Shake Sort ou TimSort;
- propor outras modificações ou experimentos relacionados ao desempenho dos algoritmos.

Essas extensões são **opcionais** e não substituem os requisitos básicos do TP1.

15. Observações gerais

O docente estará à disposição dos grupos para discutir dúvidas sobre a implementação dos algoritmos, o planejamento dos experimentos, a análise dos resultados e a elaboração do documento técnico.

Recomenda-se que os grupos discutam previamente a metodologia do experimento com o docente, especialmente antes de iniciar a coleta definitiva dos dados.

O objetivo central do TP1 é desenvolver a capacidade de **implementar, experimentar, medir, analisar e comunicar resultados relacionados a algoritmos**, articulando a prática de programação com os conceitos teóricos estudados no componente.